

Beginner's Guide To TypeScript + React Integration

From TypeScript Basics to React Integration

Created by Yogesh Chavan

yogeshchavan.dev

<https://www.linkedin.com/in/yogesh-chavan97/>

Table of Contents

Section 1: Introduction to TypeScript

1. What is TypeScript?	4
2. Why TypeScript over JavaScript?	4
3. TypeScript vs JavaScript	6
4. Installing TypeScript	6
5. Setting up TypeScript with Node.js	7
6. Your First TypeScript Program	8
7. Understanding the Compilation Process	8

Section 2: TypeScript Basics

1. Basic Types (string, number, boolean)	11
2. Type Inference	11
3. Arrays and Tuples	12
4. Enums	12
5. any, unknown, and never	14
6. void Type	15
7. Type Assertions	16
8. Literal Types	17

Section 3: Functions in TypeScript

1. Function Type Annotations	18
2. Optional & Default Parameters	19
3. Rest Parameters	21
4. Function Overloading	22
5. Arrow Functions	24
6. Callback Function Types	24

Section 4: Objects & Custom Types

1. Object Type Annotations	26
2. Type Aliases	27

3. Interfaces	28
4. Optional & Readonly Properties	28
5. Extending Interfaces	30
6. Intersection Types	30
7. Index Signatures	32

Section 5: Advanced Types

1. Union Types	33
2. Type Guards	33
3. typeof Type Guard	33
4. instanceof Type Guard	34
5. Discriminated Unions	35
6. Utility Types: Partial, Required, Pick, Omit, Record	35
7. Mapped Types	37
8. Conditional Types	37

Section 6: Generics

1. What Are Generics?	39
2. Generic Functions	39
3. Generic Interfaces	41
4. Generic Classes	41
5. Generic Constraints	43
6. keyof with Generics	43
7. Default Generic Types	46

Section 7: TypeScript with React

1. Setting Up React with TypeScript	47
2. React and TypeScript Basics	47
3. Three Ways of Defining Prop Types	50
4. Working with useState	52
5. Working with useEffect	55
6. Handling Events	57
7. Building a Complete Application	58
8. Working with useRef	61
9. Working with Context and Custom Hooks	62
10. Working with useReducer Hook	65

1. Introduction to TypeScript

What is TypeScript?

TypeScript is a strongly typed programming language that builds on JavaScript, giving you better tooling at any scale. Developed and maintained by Microsoft, TypeScript adds optional static typing and class-based object-oriented programming to JavaScript. Since TypeScript is a superset of JavaScript, existing JavaScript programs are also valid TypeScript programs.

TypeScript code cannot run directly in browsers or Node.js - it must first be compiled (transpiled) to JavaScript. This compilation step is where TypeScript catches type errors, helping you find bugs before your code runs.

Why TypeScript over JavaScript?

While JavaScript is powerful and flexible, it has limitations when building large-scale applications. TypeScript addresses these limitations:

- **Static Type Checking** - Catch errors at compile time rather than runtime
- **Better IDE Support** - Enhanced autocompletion, navigation, and refactoring
- **Improved Code Readability** - Type annotations serve as documentation
- **Enhanced Refactoring** - Safely rename symbols and restructure code
- **Latest ECMAScript Features** - Use modern features with backward compatibility

TypeScript vs JavaScript

Feature	JavaScript	TypeScript
Type System	Dynamic typing	Static typing (optional)
Compilation	Interpreted directly	Compiles to JavaScript
Error Detection	Runtime errors	Compile-time errors
IDE Support	Basic	Enhanced (IntelliSense)
Interfaces	Not supported	Fully supported
Generics	Not supported	Fully supported

Installing TypeScript

Step 1: Install Node.js

Before using TypeScript, you need Node.js installed on your machine. Follow these steps:

1. Visit the official Node.js website: <https://nodejs.org>
2. Download the LTS (Long Term Support) version for your operating system
3. Run the installer and follow the installation wizard
4. Verify the installation by opening a terminal and running:

```
1 node --version
2 # Should output: v20.x.x or higher
3
4 npm --version
5 # Should output: 10.x.x or higher
```

Step 2: Install TypeScript Globally

Once Node.js is installed, you can install TypeScript globally using npm:

```
1 npm install -g typescript
```

Step 3: Verify TypeScript Installation

```
1 tsc --version
2 # Should output: Version 5.x.x
```

Tip: You can also install TypeScript locally in a project using `npm install typescript --save-dev`. This is recommended for projects to ensure consistent versions across team members.

Setting up TypeScript with Node.js

To set up a new TypeScript project, create a directory and initialize it:

```
1 mkdir my-typescript-project
2 cd my-typescript-project
3 npm init -y
4 npm install typescript --save-dev
```

Create a TypeScript configuration file (tsconfig.json):

```
1 npx tsc --init
```

Here's a recommended configuration for beginners:

```
1 {
2   "compilerOptions": {
3     "target": "ES2020",
4     "module": "commonjs",
5     "strict": true,
6     "esModuleInterop": true,
7     "skipLibCheck": true,
8     "outDir": "./dist",
9     "rootDir": "./src"
10  },
11  "include": ["src/**/*.ts"],
12  "exclude": ["node_modules"]
13 }
```

Your First TypeScript Program

Create a file called `hello.ts` in your `src` folder:

```
1 // src/hello.ts
2 function greet(name: string): string {
3     return "Hello, " + name + "!";
4 }
5
6 const message: string = greet("TypeScript");
7 console.log(message);
```

Notice the type annotations: `name: string` specifies the parameter type, and `: string` after the parentheses specifies the return type.

Understanding the Compilation Process

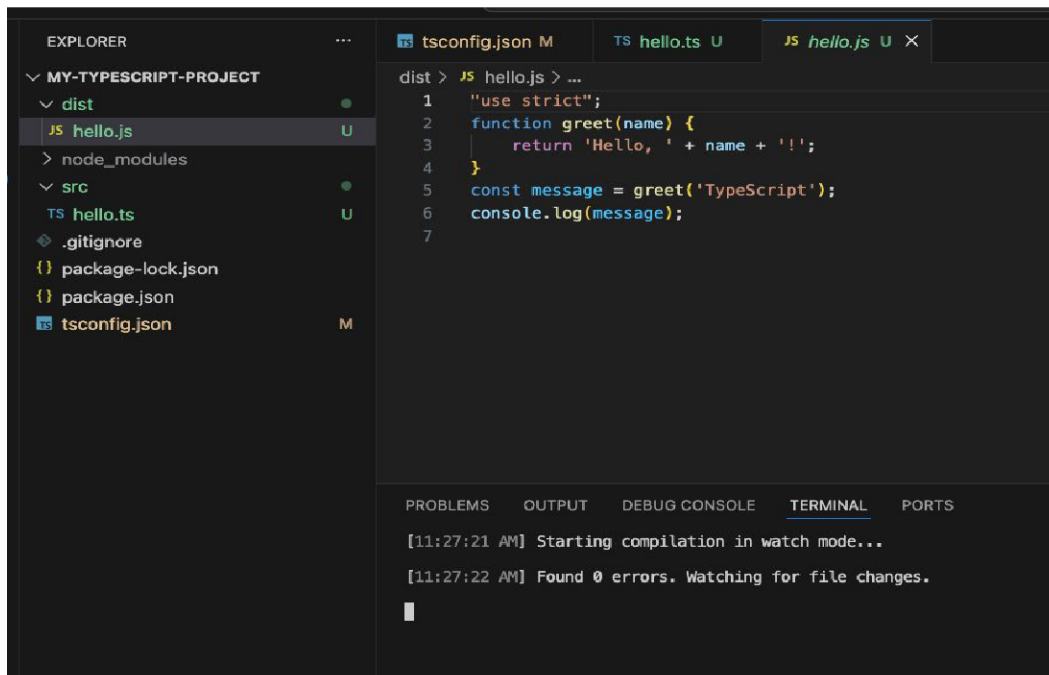
TypeScript must be compiled to JavaScript before it can run:

```
1 # Compile a single file
2 tsc src/hello.ts
3
4 # Compile using tsconfig.json
5 tsc
6
7 # Watch mode - recompile on changes
8 tsc --watch
```

When you run the `tsc` command (or `tsc --watch` for continuous compilation), TypeScript creates a new `dist` folder containing the compiled JavaScript files. You'll see your `hello.ts` file transformed into `hello.js` with all type annotations removed:

```
1 // dist/hello.js (compiled output)
2 "use strict";
3 function greet(name) {
4     return 'Hello, ' + name + '!';
5 }
6 const message = greet('TypeScript');
7 console.log(message);
```

Here's what the project structure looks like in VS Code after compilation:



Notice in the terminal that the TypeScript compiler found 0 errors and is watching for file changes. The `dist` folder now contains the compiled `hello.js` file.

Note: All type annotations are removed during compilation. Types only exist at development and compile time - they help catch errors but don't affect runtime behavior.

Running the Compiled Code

Now that your TypeScript code has been compiled to JavaScript, you can run it using Node.js. Execute the compiled JavaScript file from the `dist` folder:

```
1 # Run the compiled JavaScript file
2 node dist/hello.js
```

You should see the following output in your terminal:

```
1 Hello, TypeScript!
```

Congratulations! You've successfully written, compiled, and executed your first TypeScript program. The workflow is: write `.ts` files → compile with `tsc` → run the output `.js` files with `node`.

Tip: You can also use ts-node to run TypeScript files directly without manual compilation: `npm install -g ts-node`, then run `ts-node src/hello.ts`. This is useful during development.

2. TypeScript Basics

Basic Types (string, number, boolean)

TypeScript provides several basic types that form the foundation of the type system:

```
1 // String type
2 let firstName: string = "John";
3 let lastName: string = 'Doe';
4
5 // Number type (includes integers and floats)
6 let age: number = 30;
7 let price: number = 99.99;
8 let hex: number = 0xf00d;
9
10 // Boolean type
11 let isActive: boolean = true;
12 let isCompleted: boolean = false;
```

Type Inference

TypeScript can automatically infer types based on assigned values:

```
1 // TypeScript infers the type automatically
2 let name = "Alice";           // inferred as string
3 let count = 42;               // inferred as number
4 let isValid = true;          // inferred as boolean
5
6 // Type is locked after inference
7 name = "Bob";                 // OK
8 name = 123;                   // Error: Type 'number' is not
9                               // assignable to type 'string'
```

Best Practice: Use explicit types when the initial value doesn't clearly indicate the intended type, or when declaring variables without immediate initialization.

Arrays and Tuples

TypeScript provides two ways to define arrays, and introduces tuples for fixed-length arrays:

```
1 // Arrays - two syntax options
2 let numbers: number[] = [1, 2, 3, 4, 5];
3 let names: Array<string> = ["Alice", "Bob", "Charlie"];
4
5 // Mixed arrays
6 let mixed: (string | number)[] = [1, "two", 3];
7
8 // Tuples - fixed length with specific types
9 let person: [string, number] = ["Alice", 30];
10 let coordinate: [number, number, number] = [10, 20, 30];
11
12 // Accessing tuple elements
13 console.log(person[0]);    // "Alice" (string)
14 console.log(person[1]);    // 30 (number)
```

Enums

Enums define a set of named constants, making code more readable:

```

1  // Numeric enum (auto-increments from 0)
2  enum Direction {
3      Up,          // 0
4      Down,        // 1
5      Left,         // 2
6      Right        // 3
7  }
8
9  // Numeric enum with custom values
10 enum StatusCode {
11     OK = 200,
12     BadRequest = 400,
13     NotFound = 404,
14     ServerError = 500
15 }
16
17 // String enum
18 enum Color {
19     Red = "RED",
20     Green = "GREEN",
21     Blue = "BLUE"
22 }
23
24 // Using enums
25 let direction: Direction = Direction.Up;
26 let status: StatusCode = StatusCode.OK;

```

Using 'as const' Instead of Enums

While enums are useful, modern TypeScript development often favors using `as const` assertions instead. The `as const` assertion tells TypeScript to infer the most specific type possible, making values readonly and literal types.

Why use 'as const' over enums?

- **Tree-shaking friendly** - Regular objects with 'as const' can be tree-shaken by bundlers, while enums cannot
- **No runtime overhead** - 'as const' objects don't generate extra JavaScript code like enums do
- **Better type inference** - Works seamlessly with TypeScript's type system
- **Simpler JavaScript output** - The compiled code is just a plain object

```

1  // Using 'as const' instead of enums
2  const Direction = {
3      Up: "UP",
4      Down: "DOWN",
5      Left: "LEFT",
6      Right: "RIGHT"
7  } as const;
8
9  // Create a type from the object values
10 type Direction = typeof Direction[keyof typeof Direction];
11 // Type is: "UP" | "DOWN" | "LEFT" | "RIGHT"
12
13 // Usage
14 let move: Direction = Direction.Up; // OK
15 move = "UP"; // Also OK
16 move = "DIAGONAL"; // Error!
17
18 // Another example with status codes
19 const StatusCode = {
20     OK: 200,
21     BadRequest: 400,
22     NotFound: 404,
23     ServerError: 500
24 } as const;
25
26 type StatusCode = typeof StatusCode[keyof typeof StatusCode];
27 // Type is: 200 | 400 | 404 | 500

```

Best Practice: For new projects, prefer 'as const' objects over enums. They provide the same benefits with better bundle size and simpler JavaScript output. Use enums only when you need reverse mapping (numeric enums) or when working with legacy codebases.

any, unknown, and never

TypeScript provides special types for handling dynamic values:

```

1  // any - opts out of type checking (avoid when possible)
2  let flexible: any = 4;
3  flexible = "string";      // OK
4  flexible = true;          // OK
5  flexible.anything();      // OK (but risky!)
6
7  // unknown - type-safe alternative to any
8  let uncertain: unknown = 4;
9  uncertain = "string";     // OK
10
11 // Must check type before using
12 if (typeof uncertain === "string") {
13     console.log(uncertain.toUpperCase()); // OK
14 }
15
16 // never - represents values that never occur
17 function throwError(message: string): never {
18     throw new Error(message);
19 }
20
21 function infiniteLoop(): never {
22     while (true) {}
23 }

```

Tip: Prefer 'unknown' over 'any' when you don't know the type. It forces type checking before use, making your code safer.

void Type

The void type represents the absence of a return value:

```

1  // Function that doesn't return anything
2  function logMessage(message: string): void {
3      console.log(message);
4  }
5
6  // Arrow function with void return
7  const printNumber = (num: number): void => {
8      console.log(num);
9  };

```

Type Assertions

Type assertions tell TypeScript you know more about a value's type:

```
1 // Two syntax options
2 let someValue: unknown = "Hello, TypeScript!";
3
4 // Angle-bracket syntax
5 let strLength1: number = (<string>someValue).length;
6
7 // "as" syntax (required in React - JSX)
8 let strLength2: number = (someValue as string).length;
9
10 // Working with DOM elements
11 const input = document.getElementById("myInput") as HTMLInputElement;
12 input.value = "Hello!";
13
14 // Non-null assertion
15 function getValue(arr: number[], index: number): number {
16     return arr[index]!; // Assert it won't be undefined
17 }
```

The `!` operator is called the **non-null assertion operator**. It tells TypeScript that you are certain the value will not be null or undefined, even though TypeScript thinks it might be. In the example above, `arr[index]` could potentially be undefined if the index is out of bounds, but using `!` tells TypeScript "trust me, this value exists."

```
1 // More examples of non-null assertion (!)
2 const button = document.getElementById("submit")!;
3 // Without !, TypeScript thinks button could be null
4
5 // Use when you're certain a value exists
6 interface User {
7     name: string;
8     email?: string; // optional property
9 }
10
11 function sendEmail(user: User) {
12     // We checked elsewhere that email exists
13     const email = user.email!; // Assert it's not undefined
14     console.log("Sending to:", email);
15 }
```


Warning: Use the non-null assertion (!) sparingly. It bypasses TypeScript's null checks, so incorrect usage can lead to runtime errors. Prefer proper null checks (if statements or optional chaining ?.) when possible.

Literal Types

Literal types specify exact values a variable can hold:

```

1 // String literal types
2 type Direction = "north" | "south" | "east" | "west";
3 let heading: Direction = "north"; // OK
4 heading = "northeast"; // Error!
5
6 // Numeric literal types
7 type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;
8 let roll: DiceRoll = 4; // OK
9 roll = 7; // Error!
10
11 // In function parameters
12 function move(direction: "up" | "down" | "left" | "right") {
13     console.log(`Moving ${direction}`);
14 }
15
16 move("up"); // OK
17 move("diagonal"); // Error!

```

3. Functions in TypeScript

Function Type Annotations

TypeScript allows type annotations on function parameters and return values:

```
1 // Basic function with types
2 function add(a: number, b: number): number {
3     return a + b;
4 }
5
6 // Function type as a variable
7 let multiply: (x: number, y: number) => number;
8 multiply = function(x, y) {
9     return x * y;
10 };
11
12 // Type alias for function types
13 type MathOperation = (a: number, b: number) => number;
14
15 const subtract: MathOperation = (a, b) => a - b;
16 const divide: MathOperation = (a, b) => a / b;
17
18 // Function with object parameter
19 function printUser(user: { name: string; age: number }): void {
20     console.log(`${user.name} is ${user.age} years old`);
21 }
```

What Happens When You Pass Wrong Types?

TypeScript catches type mismatches at compile time, preventing runtime errors before your code even runs:

```
1 // Calling functions with wrong types
2 add(5, 10);           // OK: returns 15
3 add("5", 10);         // Error: Argument of type 'string' is not
4                       // assignable to parameter of type 'number'
5
6 multiply(3, 4);        // OK: returns 12
7 multiply(3, "4");      // Error: Argument of type 'string' is not
8                       // assignable to parameter of type 'number'
9
10 subtract(10, 5);      // OK: returns 5
11 subtract(10);         // Error: Expected 2 arguments, but got 1
12
13 printUser({ name: "Alice", age: 30 }); // OK
14 printUser({ name: "Bob" });           // Error: Property 'age' is
15                                       // missing in type '{ name: string; }'
16 printUser("Alice");                   // Error: Argument of type 'string'
17                                       // is not assignable to parameter
```

Key Benefit: These errors appear in your IDE as you type and during compilation - not at runtime. This is one of TypeScript's biggest advantages: catching bugs before your code runs!

Optional & Default Parameters

Functions can have optional (?) and default parameter values:

```

1  // Optional parameters (must come after required)
2  function greet(name: string, greeting?: string): string {
3      if (greeting) {
4          return `${greeting}, ${name}!`;
5      }
6      return `Hello, ${name}!`;
7  }
8
9  greet("Alice");           // "Hello, Alice!"
10 greet("Bob", "Hi");       // "Hi, Bob!"
11
12 // Default parameters
13 function createUser(
14     name: string,
15     role: string = "user",
16     active: boolean = true
17 ) {
18     return { name, role, active };
19 }
20
21 createUser("Alice");      // default role & active
22 createUser("Bob", "admin"); // default active
23 createUser("Charlie", "mod", false); // all specified

```

Important: In TypeScript, every function parameter must have its type explicitly defined. Unlike JavaScript, you cannot leave parameters untyped. This requirement ensures type safety throughout your codebase and enables the compiler to catch type-related errors early in development.

```

1  // Every parameter needs a type annotation
2  function process(name: string, count: number, active: boolean) {
3      // All parameters have explicit types
4  }
5
6  // This would cause an error in TypeScript:
7  // function process(name, count, active) { }
8  // Error: Parameter 'name' implicitly has an 'any' type
9
10 // Even callback parameters need types
11 function fetchData(callback: (data: string) => void) {
12     callback("result");
13 }

```

Rest Parameters

Rest parameters accept any number of arguments as an array:

```
1 // Rest parameters with type annotation
2 function sum(...numbers: number[]): number {
3     return numbers.reduce((total, n) => total + n, 0);
4 }
5
6 sum(1, 2);           // 3
7 sum(1, 2, 3, 4, 5); // 15
8
9 // Rest with other parameters
10 function buildName(first: string, ...rest: string[]): string {
11     return first + " " + rest.join(" ");
12 }
13
14 buildName("John", "Paul", "Smith"); // "John Paul Smith"
15
16 // Spread in function calls
17 const nums: number[] = [1, 2, 3];
18 sum(...nums); // 6
```

Understanding Spread in Function Calls

The spread operator (`...`) allows you to expand an array into individual arguments when calling a function. This is the opposite of rest parameters - while rest parameters collect multiple arguments into an array, spread expands an array into separate arguments.

```

1 // Without spread - you'd have to pass each element manually
2 const numbers = [10, 20, 30];
3 sum(numbers[0], numbers[1], numbers[2]); // 60 - tedious!
4
5 // With spread - the array is expanded into individual arguments
6 sum(...numbers); // 60 - much cleaner!
7
8 // How it works:
9 // sum(...numbers) is equivalent to sum(10, 20, 30)
10
11 // Combining arrays with spread
12 const moreNumbers = [40, 50];
13 sum(...numbers, ...moreNumbers); // 150
14
15 // Spread with Math functions
16 const values = [5, 10, 3, 8, 1];
17 Math.max(...values); // 10
18 Math.min(...values); // 1
19
20 // TypeScript ensures type safety with spread
21 const strings = ["a", "b", "c"];
22 sum(...strings); // Error: Argument of type 'string' is not
23                  // assignable to parameter of type 'number'

```

Tip: The spread operator is especially useful when working with arrays of unknown length, or when you want to pass array elements as individual function arguments without modifying the original function.

Function Overloading

Function overloading lets you define multiple signatures for one function:

```
1 // Overload signatures
2 function format(value: string): string;
3 function format(value: number): string;
4 function format(value: Date): string;
5
6 // Implementation
7 function format(value: string | number | Date): string {
8     if (typeof value === "string") {
9         return value.toUpperCase();
10    } else if (typeof value === "number") {
11        return value.toFixed(2);
12    } else {
13        return value.toISOString();
14    }
15 }
16
17 format("hello");           // "HELLO"
18 format(3.14159);          // "3.14"
19 format(new Date());        // "2024-01-15T..."
```